

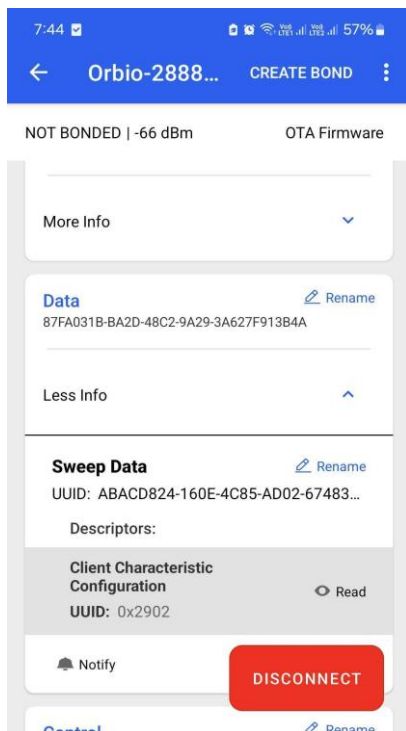
Orbio Biosensor BLE interface v2

BLE Interface

The device will advertise with name Orbio-abcdef where abcdef are 6 characters that come from the unique Bluetooth ID of the device.

a) There is a **BLE** service for the **Data** interface to the device. This service has UUID: 87fa031b-ba2d-48c2-9a29-3a627f913b4a. Under this service there is the following characteristic:

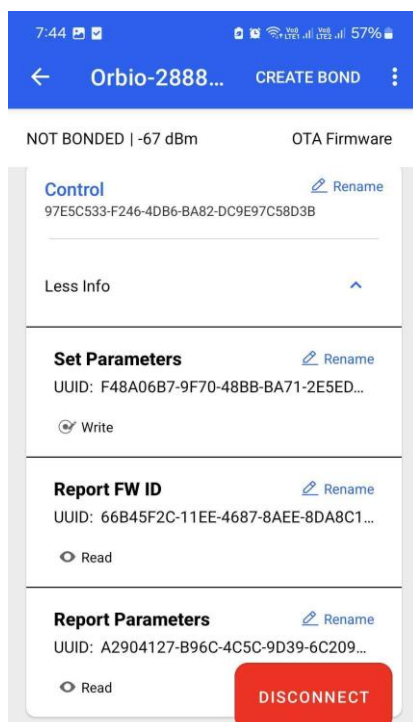
1. Sweep Data:
 - a. **Notification** characteristic which has to be enabled by the App after BLE connection is established, for data transfer to be enabled
 - b. UUID abacd824-160e-4c85-ad02-67483ad42060
 - c. Upto 240 bytes of data



b) There is a **BLE** service for the **Control** interface to the device. This service has UUID: 97e5c533-f246-4db6-ba82-dc9e97c58d3b. Under this service there are the following characteristics:

1. Set Parameters:
 - a. **Write** characteristic to set a parameter as below, as comma separated ASCII strings, with the first parameter specifying which parameter is being set. This is explained via examples.
 - b. Sweep commands start from 10, PPG commands from 20, Accel commands from 30, and Misc commands from 40
 - c. **Start Sweep**: e.g. 10 (no parameters)
 - d. **Stop Sweep**: e.g. 11 (no parameters)

- e. **IQ Offsets:** e.g. 12,-100,200 sets I Offset to -100 mV and Q Offset to +200 mV (each from -775 mV to +775 mV in steps of 1 mV)
 - f. **Set Frequency:** e.g. 13,830 sets the frequency to 830 MHz (frequency from 700 MHz to 1080 MHz)
 - g. **Run Calibration:** e.g. 14 (no parameters). **When to run this and how to use the results or send to the client is TBD.**
 - h. **Set LNA & VGA gains:** e.g. 15,1,7 sets LNA gain to 1 * -6 dB and VGA gain to 7 * -6 dB (LNA gain from 0 to 3 and VGA gain from 0 to 7 both in -6 dB steps)
 - i. **Sweep/Sleep timings:** e.g. 16,60,600 sets sweep for 60 seconds and interval between sweeps to 600 seconds (in steps of 1 second, allowed range for both fields is from 10 to 6000 seconds)
 - j. **Set trigger frequency:** e.g. 17,0 sets the TP1 to pulse low after a new frequency has been programmed. For pulsing TP1 only for a specific frequency N from 1 to 20, send 17,N. The default value is 0 i.e. pulse TP1 at all frequencies.
 - k. **Start Accel:** e.g. 30 (no parameters)
 - l. **Stop Accel:** e.g. 31 (no parameters)
 - m. **Start PPG:** e.g. 20 (no parameters)
 - n. **Stop PPG:** e.g. 21 (no parameters)
 - o. **Print samples:** e.g. 40,1 (enable printing samples) or 40,0 (disable printing samples)
 - p. UUID f48a06b7-9f70-48bb-ba71-2e5edc537e59
2. Report FW ID
 - a. **Read** characteristic to get the firmware version (one byte)
 - b. UUID 66b45f2c-11ee-4687-8aee-8da8c1155384
 3. Report Parameters
 - a. **Read** characteristic to get the current parameters. This will be in ASCII form, with three comma separated parameters in this order: PLL gain (in steps of 1 dB), Active Sweep Time (in seconds), Interval between Sweeps (in seconds)
 - b. UUID a2904127-b96c-4c5c-9d39-6c209cd4e92c



BLE Data Format

Sweep data:

One full sweep worth of data will be sent after collection. This will consist of 32 samples of I & Q each, 16-bits per I and per Q sample each, at 39 frequencies (700 MHz to 1080 MHz in steps of 10 MHz). So, one sweep data frame will contain $32 * 2 * 2 * 39 = 4992$ Bytes arranged as shown here.

Note that a sweep's worth of data i.e. a frame will be split over a number of BLE packets as the maximum BLE packet size we are sending is 240 bytes which is just within the BLE MTU size of 251 which is negotiated with the client.

The header field 0xAABBCC marks the start of a new frame and should be used as a safety check for previous frame being received fully.

Similarly, the frame length field should be used at the end of a frame to check if the frame was received fully.

Frame Header	0xAA 0xBB 0xCC	
Frame info	1-byte frame type (1=sweep, 2=PPG, 3=Accel)	
	4-byte timestamp	
	2-byte frame length	
Freq1	I1	Q1
	I2	Q2
	.	
	.	
	.	
	I32	Q32
Freq2	I1	Q1
	I2	Q2
	I3	Q3
	.	
	.	
	I32	Q32
.	.	
.	.	
.	.	
Freq 39	I1	Q1
	I2	Q2
	I3	Q3
	.	
	.	
	I32	Q32

PPG data:

One PPG FIFO interrupt worth of data will be sent after collection. This will consist of 3 Bytes per PPG sample, with the top 4 bits indicating the Measurement number tag (from 1 to 8), and remaining 20 bits representing the signed sample value in Big Endian format. Typically, in steady state, we can expect around 72 samples of 3 bytes each per data frame i.e. 216 bytes, but if there are tag or syncing errors we can expect less or more as well. As this is less than the BLE limit of 240 bytes in a packet, this will typically come in a single packet, accounting for the 10-byte header. So, the data frame can be of variable length and must be determined and error-checked by using the frame length & fixed frame header position in data.

A plot of the above data will therefore contain 8 different waveforms, one per measurement.

Frame Header	0xAA 0xBB 0xCC		
Frame info	1-byte frame type (2=PPG) 4-byte timestamp 2-byte frame length		
sample1 (3 bytes = 24 bits)	tag = meas number (4 bits)	4 MS bits	8 + 8 LS bits
sample2 (3 bytes = 24 bits)	tag = meas number (4 bits)	4 MS bits	8 + 8 LS bits
sample3 (3 bytes = 24 bits)	.		8 + 8 LS bits
	.		
	.		
samplen (3 bytes = 24 bits)	tag = meas number (4 bits)	4 MS bits	8 + 8 LS bits
All 20-bit data in signed Big Endian			

Accel data:

One second worth of Accel data will be sent after collection. This will typically be 24 to 27 XYZ axis data of 2 bytes per axis, so upto $27 * 3 * 2 = 162$ Bytes per frame, as shown here. As this is less than the BLE limit of 240 bytes in a packet, this will come in a single packet, accounting for the 10-byte header. The data is presented in Little Endian format for each axis. The data frame can be of variable length and must be determined and error-checked by using the frame length & fixed frame header position in data.

A plot of the above data will therefore contain 3 different waveforms, one per axis.

Frame Header	0xAA 0xBB 0xCC		
Frame info	1-byte frame type (3=Accel) 4-byte timestamp 2-byte frame length		
sample1 (2 bytes per axis)	X1	Y1	Z1
sample2 (2 bytes per axis)	X2	Y2	Z2
sample3 (2 bytes per axis)	.	.	.
	.	.	.
	.	.	.
samplen (2 bytes per axis)	Xn	Yn	Zn
All 2-byte data in Little Endian			

Command Line Interface (CLI) Controls

- help: shows available commands, explanation and default settings
- dosweep: Trigger sweep manually
- stop_sweep: Stop sweep
- set_freq: Set frequency
- cal_offsets: Calibrate I/Q offsets
- set_lna_vga_gain: Set LNA and VGA gains in steps of -6 dB
- set_sweep_timing: set sweep and sleep timings in seconds
- set_iq_offsets: set I/Q DC offsets in mV
- start_accel: Start accel sampling
- stop_accel: Stop accel mode
- start_ppg: Start PPG sampling
- stop_ppg: Stop PPG mode
- A few more commands set_trigfreq, msweep, print_samples, print_unit_id, print_fw_ver are available and used during development

Usage

By default, the device will perform sweeps with 30 seconds sweep and 10 minutes sleep. Just before starting a sweep, BLE advertisement is enabled and if a client connects and enables the notification

then the sweep data is transferred to the client after each sweep.

The CLI can be used to change the required settings from default, like sweep timing, PLL gain, IQ DC offsets.

At compile time we can decide whether to keep either/both of CLI and BLE active during sleep to change settings via commands.

Data Output

print_samples 1 (to enable I/Q sample printing) can be used to view values. There will be 32 I/Q pairs per frequency. The print has been kept simple, with comma separated values for I & Q only, to be able to plot on SerialPlot if needed.

The output is in 15-bit signed format i.e. sample values from 0 to 32767.

Over-The-Air Device Firmware Update (OTADFU)

OTADFU testing is TBD.